Learning Fine-Tuning: The 20% That Matters

1 First: Do You Even Need Fine-Tuning?

Fine-tuning teaches a model a new **behavior, format, or style**. It does NOT reliably teach it new **facts** — that is what RAG (the previous document) is for. Decide before you start:

- **Use RAG** when the model needs up-to-date or private *knowledge*.
- **Use fine-tuning** when you need a consistent *output format*, a specific *tone/persona*, a narrow *task* the base model does poorly, or to make a smaller model imitate a larger one.
- **Often: use both** — a fine-tuned model that answers in your style, fed retrieved facts.

Try prompt engineering first; it is free. Fine-tune only when prompting plateaus.

2 The Mental Model

Fine-tuning continues training a **pretrained** model on your own dataset of examples so its weights shift toward your task. The naive approach (full fine-tuning) updates every weight and is extremely expensive: fully fine-tuning a 7B model can demand well over 140 GB of VRAM — multiple A100/H100 GPUs.

The practical 20% that everyone actually uses is **LoRA / QLoRA**, which makes this feasible on a single consumer GPU.

3 LoRA and QLoRA (The Core Idea)

- **LoRA (Low-Rank Adaptation):** freeze all the original weights, inject tiny trainable "adapter" matrices, and train only those. This cuts trainable parameters by over 90%, so memory and compute drop dramatically. The base model is untouched; you produce a small adapter file.
- **QLoRA:** LoRA *plus* loading the frozen base model in 4-bit precision (quantization). This shrinks the base model's memory footprint so a 7B model fits on one consumer GPU, with accuracy close to full LoRA.

Takeaway: QLoRA = 4-bit base model + small trainable adapters. It is the default starting point today.

4 The Standard Toolkit

Four Hugging Face libraries do the heavy lifting:

- `transformers` — loads the base model and tokenizer.

- `peft` — implements LoRA/QLoRA (the adapters).
- `trl` — provides `SFTTrainer`, which wraps the whole training loop.
- `bitsandbytes` — does the 4-bit quantization for QLoRA.

```
pip install transformers peft trl bitsandbytes datasets accelerate
```

Version warning: `SFTTrainer`'s API has changed significantly across `trl` versions. Always check the docs for the version you install — argument names move between the trainer and a config object.

5 Step 1 — The Dataset (Most Important Part)

Supervised fine-tuning (SFT) trains on consistent **prompt–response pairs**. Garbage data gives a garbage model; quality and consistent formatting matter far more than quantity. A common format is JSONL, one example per line:

```
{"prompt": "Summarize: <text>", "completion": "<summary>"}
```

Or the increasingly standard **chat** format:

```
{"messages": [
  {"role": "user", "content": "Translate to French: Hello"},
  {"role": "assistant", "content": "Bonjour"}
]}
```

```
from datasets import load_dataset
dataset = load_dataset("json", data_files="train.jsonl", split="train")
```

Rules: keep the format identical across all examples; cover the variety you expect at inference; a few hundred to a few thousand high-quality examples often beats tens of thousands of noisy ones; hold some out for evaluation.

6 Step 2 — Load the Model in 4-bit (QLoRA)

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer,
    BitsAndBytesConfig

model_name = "mistralai/Mistral-7B-v0.1"

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",            # recommended 4-bit type
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,
)

model = AutoModelForCausalLM.from_pretrained(
    model_name, quantization_config=bnb_config, device_map="auto"
)
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token    # common fix for missing pad token
```

7 Step 3 — Configure LoRA

```
from peft import LoraConfig

lora_config = LoraConfig(
    r=16,                    # rank: size of the adapters (8-64 typical)
    lora_alpha=32,           # scaling; common to set alpha = 2 * r
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],  # which layers get adapters
)
```

The knobs that matter:

- `r` (rank) — bigger = more capacity to learn, more compute. Start at 16.
- `lora_alpha` — scaling factor; a common heuristic is $2 \times r$.
- `target_modules` — which weight matrices get adapters (the attention projections are the usual choice).

8 Step 4 — Train with SFTTrainer

`SFTTrainer` hands you a clean loop: give it the model, LoRA config, dataset, and training arguments. (Recent `trl` puts most settings in an `SFTConfig` object.)

```
from trl import SFTTrainer, SFTConfig

training_args = SFTConfig(
    output_dir="./out",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,    # effective batch = 4 * 4 = 16
    learning_rate=2e-4,               # higher than full FT; typical for LoRA
    logging_steps=10,
    save_steps=100,                   # checkpoint regularly
    bf16=True,
    max_seq_length=512,
    packing=True,                     # pack short examples for efficiency
)

trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=dataset,
```

```
    peft_config=lora_config,          # SFTTrainer applies LoRA for you
)

trainer.train()
trainer.save_model("./adapter")       # saves the small LoRA adapter only
```

9 The Hyperparameters You Will Actually Tune

- **learning rate** — LoRA tolerates higher LRs; $1e{-}4$ to $3e{-}4$ is typical.
- **epochs** — 1–3 is usual; too many causes overfitting (model memorizes, loses generality).
- **batch size + gradient accumulation** — raise accumulation if VRAM is tight (simulates a bigger batch).
- **LoRA rank `r`** — capacity vs cost.
- **max sequence length** — longer = more memory.

10 Step 5 — Use the Fine-Tuned Model

The adapter is tiny and loads on top of the base model:

```
from peft import PeftModel

base = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto")
model = PeftModel.from_pretrained(base, "./adapter")

inputs = tokenizer("Translate to French: Good morning", return_tensors="pt
    ").to(model.device)
out = model.generate(**inputs, max_new_tokens=50)
print(tokenizer.decode(out[0], skip_special_tokens=True))
```

For deployment you can **merge** the adapter into the base weights to remove the runtime overhead:

```
merged = model.merge_and_unload()
merged.save_pretrained("./merged_model")
```

11 Evaluation and Overfitting

- Watch **training loss** go down — but also check a held-out **validation set**, or the model just memorizes.
- Run a **baseline** (the untrained model) on your test prompts first, so you can prove the fine-tune actually improved things.
- Track runs with Weights & Biases or MLflow to compare configurations.
- Signs of overfitting: great on training data, worse/repetitive on new prompts → fewer epochs, more/varied data, or lower rank.

12 Beyond SFT (Awareness Level)

Supervised fine-tuning teaches imitation. To align a model to *preferences* (which of two answers is better), the next step is **preference tuning** — DPO and ORPO, also available in `trl`. You usually do SFT first, then preference tuning. Know these names; you rarely need them for a first project.

13 Practical Tips That Save Hours

- Start with a **tiny model and tiny dataset** to confirm the pipeline runs end-to-end before scaling.
- Save **checkpoints** so a crash doesn't lose everything.
- Use `packing=True` to fit more short examples per batch.
- For 2–5× faster QLoRA on consumer GPUs, look at **Unsloth**.
- Rent GPUs ad-hoc (e.g. cloud A100/H100) rather than buying hardware.

14 The Checklist for Any Fine-Tune

1. Confirm fine-tuning (not RAG/prompting) is the right tool.
2. Build a clean, consistently formatted prompt–response dataset.
3. Load the base model in 4-bit (QLoRA) with `bitsandbytes`.
4. Configure LoRA (`r`, `alpha`, `target_modules`).
5. Train with `SFTTrainer`; checkpoint and log.
6. Evaluate vs a baseline on held-out data; watch for overfitting.
7. Save the adapter; merge for deployment.

15 Practice Task

Fine-tune a small model on a tiny custom task:

1. Pick a small base model (e.g. a 1–2B model) so it runs quickly.
2. Make a 50–100 line JSONL dataset for one narrow task (e.g. "turn a sentence into a polite version").
3. Run a baseline generation on 5 test prompts and save the outputs.
4. Fine-tune with QLoRA + `SFTTrainer` for 1–3 epochs.
5. Generate on the same 5 prompts and compare before vs after.
6. Bonus: serve the merged model behind a FastAPI endpoint, in a Docker container, managed with uv.